Project Report: Parking Management System

1. Problem Statement and Solution Approach

Problem Statement: Urban areas face significant challenges in managing parking spaces efficiently. Drivers often spend excessive time searching for available spots, leading to traffic congestion and increased carbon emissions. Administrators struggle to monitor lot capacity, track vehicle check-ins, and manage payments in real-time, often relying on manual or disjointed legacy systems.

Solution Approach: The Parking Management System is a comprehensive, full-stack application designed to digitize and automate parking infrastructure.

- **For Drivers:** A seamless interface to view real-time slot availability, reserve a space, and process digital payments.
- **For Administrators:** A centralized dashboard to manage physical infrastructure (Lots, Floors, Slots) and monitor incoming/outgoing vehicles.
- **Architecture:** The system utilizes a React frontend and an Express/Node.js backend, communicating via RESTful APIs. It is backed by a robust, normalized PostgreSQL database managed through the Prisma ORM.

---

2. System Design Optimization

To ensure scalability, performance, and maintainability, the following System Design principles were applied:

1. **Decoupled Monorepo Architecture:** The project uses Turborepo to manage the frontend (`apps/web`) and backend (`apps/api`) in a single repository. This allows for shared TypeScript configurations and synchronized deployments while keeping the codebases strictly isolated.
2. **Database Normalization:** The PostgreSQL schema is strictly normalized into 8 distinct tables (Lots -> Floors -> Slots -> Bookings -> Payments). This eliminates data redundancy.
3. **ACID Transactions for Data Integrity:** Critical workflows, such as paying for a booking, execute multiple queries simultaneously. Using `prisma.$transaction()` , we ensure that updating a Payment to "completed" and updating a Booking to "confirmed" happens atomically. If one fails, the entire transaction rolls back, preventing ghost bookings.
4. **Stateless Authentication (JWT):** By using JSON Web Tokens (JWT) instead of stateful sessions, the API only needs to verify the cryptographic signature of the token, drastically reducing database read overhead on every request.

---

3. Object-Oriented Programming (OOP) Concepts

1. **Encapsulation:** Database repositories are injected into services via `private` constructors. The raw database logic is encapsulated and hidden from the outside world, preventing external tampering.

```
export class PaymentService implements IPaymentService {
  // Encapsulated dependency: external classes cannot access or manipulate this
repository directly
  constructor(private readonly paymentRepository: IPaymentRepository) {}

  public async getPaymentDetails(id: string) {
    return this.paymentRepository.findById(id); // Safe, controlled access
```

```
  }
}
```

2. **Abstraction:** Controllers interact with services through abstract interfaces (`IPaymentRepository`). They request data without knowing *how* the data is retrieved, hiding the complex SQL/Prisma implementation details.

```
// The complex Prisma database logic is completely abstracted away from the Service
layer
export interface IPaymentRepository {
  findById(id: string): Promise<Payment | null>;
  createPayment(data: CreatePaymentDTO): Promise<Payment>;
}
```

3. **Inheritance:** The `AppError` utility class inherits from the standard Node.js `Error` class to maintain stack trace behaviors while extending it with custom HTTP status codes (like `400` or `404`).

```
// Inherits standard Error trace behavior but extends it with custom HTTP context
export class AppError extends Error {
  public readonly statusCode: number;

  constructor(message: string, statusCode = 500) {
    super(message); // Calls the parent Error constructor
    this.statusCode = statusCode;
  }
}
```

4. **Polymorphism:** Repositories act polymorphically—multiple repository types can implement the same interface. The application logic behaves differently depending on which instance is passed at runtime, without needing `if/else` statements.

```
// Both classes implement IUserRepository, meaning AuthService can use either
// polymorphically without caring which exact class is being executed at runtime.
export class PrismaUserRepository implements IUserRepository {
  async getByEmail(email: string) { return prisma.user.findUnique({ where: { email
}}); }
}

export class MockUserRepository implements IUserRepository {
  async getByEmail(email: string) { return { id: "123", email, role: "driver" }; }
}
```

---

4. Design Patterns

1. Singleton Pattern

Where: `apps/api/src/lib/prisma.ts` **Why:** The `PrismaClient` instance is exported as a singleton, guaranteeing that every API request shares the same global database connection pool, preventing connection exhaustion.

```
class Prisma {
  private static instance: PrismaClient;
  public static getInstance(): PrismaClient {
    if (!this.instance) {
      this.instance = new PrismaClient({ adapter });
    }
    return this.instance;
  }
}
```

2. Strategy Pattern

Where: The Repository layer (e.g., `user.repository.ts`) **Why:** Services interact with the abstract Strategy (`IUserRepository`), allowing us to swap out the underlying database engine without touching the core API logic.

```
export interface IUserRepository {
  getByEmail(email: string): Promise<DBUser | null>;
}
export class PrismaUserRepository implements IUserRepository {
  public async getByEmail(email: string) {
    return prisma.user.findUnique({ where: { email } });
  }
}
```

3. Factory Pattern / IoC

Where: Container files (e.g., `auth.container.ts`) **Why:** The Container functions act as Abstract Factories that instantiate Repositories, inject them into Services, and inject Services into Controllers before returning the fully wired module.

```
export function createAuthModule() {
  const userRepository = new PrismaUserRepository();
  const authService = new AuthService(userRepository); // Factory wired
  return { authService };
}
```

4. Adapter Pattern

Where: `JoseTokenService` **Why:** It acts as an adapter, taking the complex methods of the external `jose` JWT library and mapping them to our internal, simplified `ITokenService` interface.

```
export class JoseTokenService implements ITokenService {
  // Adapts complex jose syntax to a simple string return
  public async sign(payload: Record<string, unknown>, type: "ACCESS" | "REFRESH"):
Promise<string> {
```

```
    const secret = type === "ACCESS" ? this.accessSecret : this.refreshSecret;
    return new SignJWT(payload).setProtectedHeader({ alg: "HS256" }).sign(secret);
  }
}
```

---

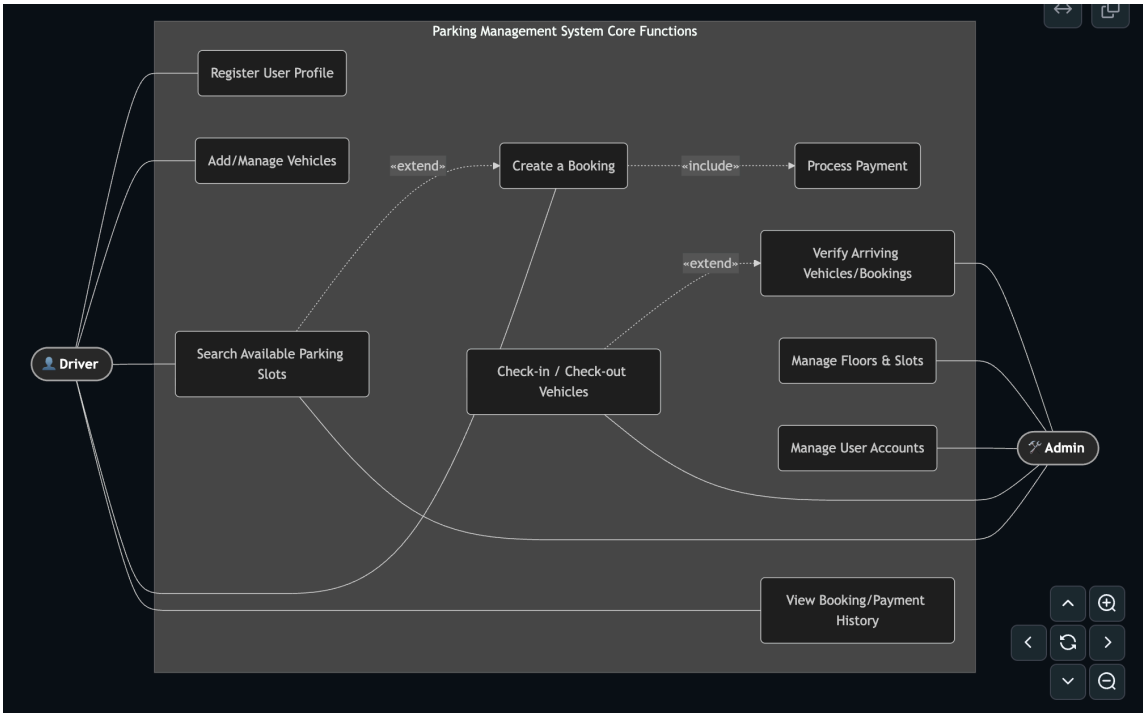
5. SOLID Principles

1. **Single Responsibility Principle (SRP):** The codebase separates concerns by layer. `Controllers` only parse HTTP requests/responses. `Services` only enforce business rules. `Repositories` only execute raw SQL/Prisma queries.
2. **Open/Closed Principle (OCP):** By relying on abstract interfaces (like `IPaymentRepository`), the services are open for extension (adding new database engines) but closed for modification.
3. **Liskov Substitution Principle (LSP):** Any class that implements `IUserRepository` (like a Mock DB for testing) can seamlessly replace `PrismaUserRepository` in the `AuthService` constructor without breaking the application.
4. **Interface Segregation Principle (ISP):** Instead of a massive, bloated `IAppService` interface, the system uses tiny, specific interfaces like `IBookingService` and `IPaymentService`.
5. **Dependency Inversion Principle (DIP):** High-level modules (Controllers/Services) do not depend on low-level modules (Prisma/SQL). Both depend on abstractions (Interfaces) via constructor Dependency Injection.

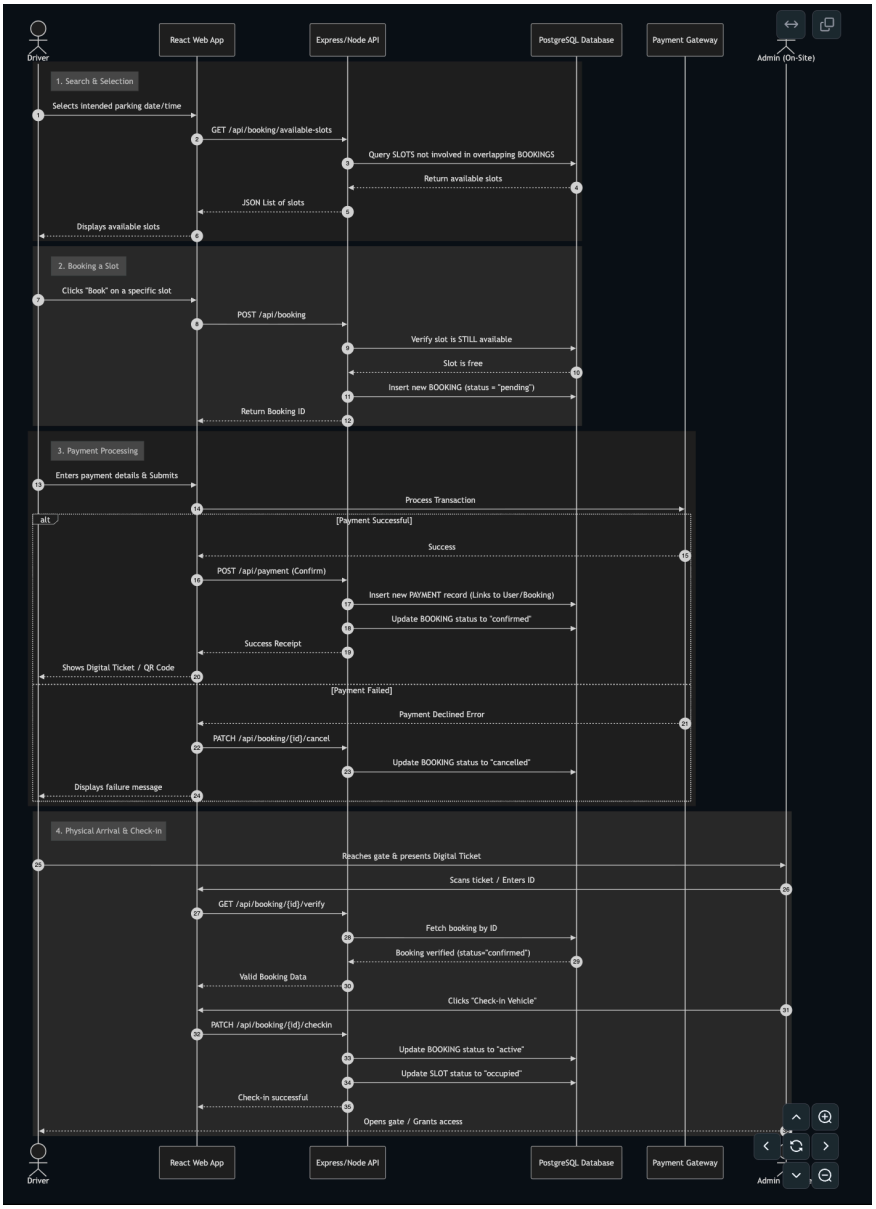
---

6. UML Diagrams

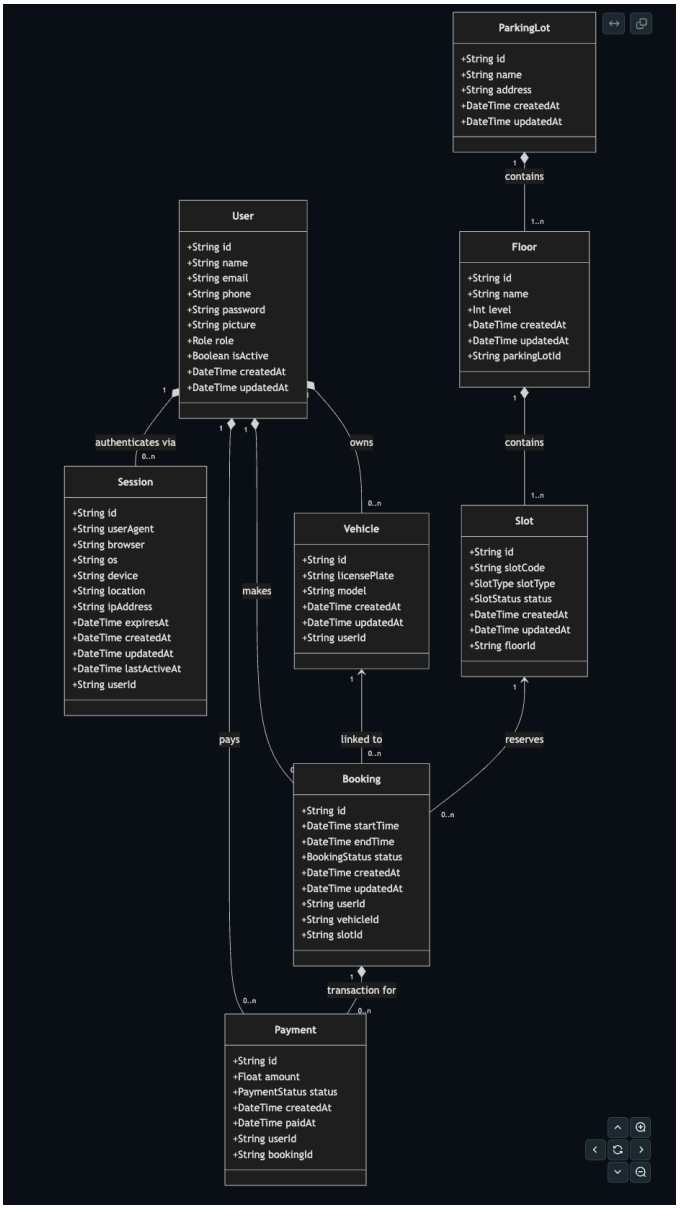
A. Use Case Diagram



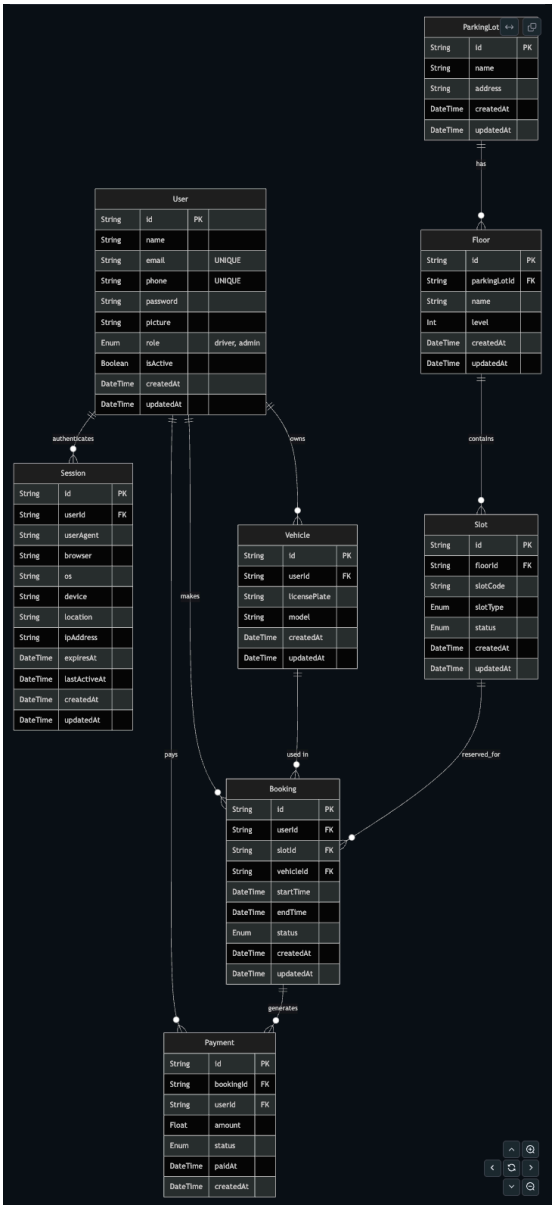
B. Sequence Diagram (Booking & Payment)



C. Class Diagram


ParkingLot
+String id
+String name
+String address
+DateTime createdAt
+DateTime updatedAt
1
contains
1..n
Floor
+String id
+String name
+Int level
+DateTime createdAt
+DateTime updatedAt
+String parkingLotId
1
contains
1..n
Slot
+String id
+String slotCode
+SlotType slotType
+SlotStatus status
+DateTime createdAt
+DateTime updatedAt
+String floorId
1
reserves
0..n
User
+String id
+String name
+String email
+String phone
+String password
+String picture
+Role role
+Boolean isActive
+DateTime createdAt
+DateTime updatedAt
1
authenticates via
0..n
Session
+String id
+String userAgent
+String browser
+String os
+String device
+String location
+String ipAddress
+DateTime expiresAt
+DateTime createdAt
+DateTime updatedAt
+DateTime lastActiveAt
+String userId
1
owns
0..n
Vehicle
+String id
+String licensePlate
+String model
+DateTime createdAt
+DateTime updatedAt
+String userId
1
linked to
0..n
1
makes
1
pays
0..n
Booking
+String id
+DateTime startTime
+DateTime endTime
+BookingStatus status
+DateTime createdAt
+DateTime updatedAt
+String userId
+String vehicleId
+String slotId
1
transaction for
0..n
Payment
+String id
+Float amount
+PaymentStatus status
+DateTime createdAt
+DateTime paidAt
+String userId
+String bookingId

D. ER Diagram



---

7. Test Cases and Results

Test ID	Scenario	Expected Result	Actual Result	Status
TC_01	Driver logs in with valid credentials	System generates JWT token	Token generated	**Pass**
TC_02	User attempts to access Admin dashboard	Middleware throws 403 Forbidden	Access Denied (403)	**Pass**
TC_03	Driver searches for slots on specific dates	Queries non-overlapping slots only	Excludes occupied slots	**Pass**
TC_04	Driver books an available slot	Creates 'pending' booking & payment	Records inserted	**Pass**

TC_05	Payment simulation succeeds	Updates Payment and Booking atomically	Tables updated	**Pass**
TC_06	Payment logic fails midway	Prisma rolls back the `$transaction`	No partial updates	**Pass**
TC_07	Driver hits payment API > 200 times	Rate Limiter blocks request	429 Too Many Requests	**Pass**